



Исходный материал

Проект

Команда разрабатывает интернет-магазин **ShopFlow**.

В системе уже работают регистрация пользователей, каталог товаров, корзина, оформление заказа, онлайн-оплата, история заказов и email-уведомления.

Backend разработан на Python. Основные функции доступны через REST API. Данные хранятся в PostgreSQL.

Frontend представляет собой веб-приложение.

Оплата выполняется через внешний платёжный сервис.

Проект уже используется реальными пользователями, поэтому существующий процесс оформления и оплаты заказа нельзя нарушать.

Текущий спринт

Продолжительность спринта:

2 недели

Рабочая ёмкость команды на этот спринт:

24 story points

Команда использует следующую относительную шкалу оценки:

1
2
3
5
8
13

Значение **1** соответствует очень небольшой и хорошо понятной работе.

Чем выше значение, тем больше предполагаемый объём работы, количество зависимостей, техническая сложность или неопределённость.

Задача с оценкой **13** считается слишком крупной для обычной задачи спринта и должна дополнительно обсуждаться или декомпозироваться.

Состав команды

В спринте участвуют два backend-разработчика, один frontend-разработчик и один QA-инженер.

Один из backend-разработчиков хорошо знает модуль заказов и платежей.

Второй backend-разработчик присоединился к проекту недавно.

Frontend-разработчик в течение первых двух дней спринта занят завершением задачи предыдущего спринта.

QA-инженер доступен весь спринт, но одновременно сопровождает исправления production-дефектов.

Story points являются общей командной оценкой и не распределяются напрямую по отдельным сотрудникам.

Эпик

Название эпика:

Самостоятельный возврат оплаченного заказа

Бизнес хочет позволить пользователю оформлять возврат заказа непосредственно из личного кабинета без обращения в службу поддержки.

Сейчас пользователь пишет в поддержку. Сотрудник вручную проверяет заказ, после чего инициирует возврат через административную систему.

Новый процесс должен сократить количество обращений в службу поддержки.

Основной пользовательский сценарий

Пользователь открывает страницу своего заказа.

Если заказ допускает возврат, пользователь видит действие:

Оформить возврат

Пользователь выбирает позиции, которые хочет вернуть.

Можно вернуть как весь заказ, так и только часть позиций.

После отправки запроса система создаёт заявку на возврат.

Пользователь должен видеть текущий статус возврата в личном кабинете.

После успешного подтверждения возврата система должна инициировать возврат денежных средств через существующий платёжный сервис.

После изменения статуса пользователь получает email-уведомление.

Известные бизнес-правила

Возврат можно оформить не позднее чем через 14 дней после получения заказа.

Нельзя вернуть товар, если количество возвращаемых единиц превышает количество купленных.

Нельзя повторно вернуть позицию, которая уже была полностью возвращена ранее.

Для одного заказа может существовать несколько частичных возвратов.

Стоимость доставки автоматически не возвращается.

Сумма возврата рассчитывается на основании фактически оплаченной стоимости позиции с учётом скидки.

Возврат денег должен выполняться только после подтверждения заявки.

Текущие статусы заказа

В системе используются статусы:

```
new
paid
processing
shipped
delivered
cancelled
```

Дата фактического получения заказа хранится в поле:

```
delivered_at
```

Для старых заказов это поле иногда может быть пустым, даже если статус равен:

```
delivered
```

Команда пока не решила, как обрабатывать такие старые записи.

Предполагаемые статусы возврата

Бизнес предварительно предлагает использовать:

```
requested
approved
rejected
refunded
```

Это пока не утверждённый технический контракт.

Не определено, нужен ли отдельный статус для ситуации, когда заявка была подтверждена, но платёжный сервис временно не смог вернуть деньги.

Платёжный сервис

В проекте уже существует интеграция с внешним платёжным сервисом.

Для обычной оплаты используется существующий клиент:

```
PaymentClient
```

В документации клиента найден метод:

```
refund(payment_id, amount)
```

Команда раньше этот метод в production не использовала.

Известно, что запрос к внешнему платёжному сервису может завершиться timeout.

Пока не выяснено, безопасно ли автоматически повторять операцию `refund()` после timeout.

Платёжный сервис поддерживает только денежную сумму возврата. Он ничего не знает о товарах или позициях заказа.

Текущая модель заказа

Упрощённо в проекте используются сущности:

```
Order
OrderItem
Payment
User
Product
```

`OrderItem` содержит количество товара и цену на момент покупки.

Скидка также сохраняется вместе с заказом, поэтому текущая цена товара в каталоге не должна использоваться при расчёте возврата.

Существующий API

Для пользователя уже существуют методы:

```
GET /orders
GET /orders/{order_id}
```

Для административной части существует отдельный API.

Архитектурная команда пока не определила окончательный набор endpoints для возвратов.

Frontend

На странице заказа сейчас отображаются данные заказа, позиции, стоимость, статус доставки и данные оплаты.

Отдельного интерфейса возвратов пока нет.

Дизайнер предоставил предварительный макет, в котором пользователь выбирает позиции чекбоксами и количество возвращаемых единиц.

После отправки формы должен появляться статус заявки.

Финальный дизайн страницы ещё может немного измениться, но основной пользовательский сценарий уже утверждён.

Email-уведомления

В проекте уже существует сервис:

```
NotificationService
```

Он используется для уведомлений о создании и отправке заказа.

Для возвратов необходимо добавить новые шаблоны.

Бизнес хочет уведомлять пользователя как минимум после создания заявки и после окончательного возврата денежных средств.

Пока не решено, нужно ли отправлять отдельное письмо при отклонении заявки.

Административная часть

Для первого релиза сложный новый интерфейс администратора создавать не хотят.

Предполагается использовать существующую административную панель.

Однако сейчас в ней нет возможности просматривать заявки на возврат и изменять их статус.

Бизнес считает, что минимальная возможность подтверждать или отклонять заявку всё равно должна попасть в первый релиз.

Требования к тестированию

Основная логика расчёта суммы возврата должна быть покрыта автоматическими тестами.

Нужно проверить полный возврат и частичный возврат.

Необходимо проверить ограничения по количеству товара.

Нужно проверить попытку возврата заказа за пределами разрешённого периода.

Интеграцию с платёжным сервисом нельзя полноценно вызывать из обычных unit-тестов.

В проекте уже используется mock внешнего платёжного клиента.

Ограничения первого релиза

Первый релиз не должен поддерживать обмен товара на другой товар.

Автоматическое создание транспортной накладной для обратной доставки пока не требуется.

Интеграция с системой склада также не входит в первый релиз.

Мобильного приложения у проекта нет, поэтому требуется только веб-интерфейс.

Поддержка разных валют пока не нужна.

Неопределённые вопросы

До начала реализации окончательно не решено, что делать с заказами, у которых отсутствует `delivered_at`.

Не определено поведение после timeout платёжного метода `refund()`.

Не утверждён окончательный набор статусов возврата.

Не решено, должно ли отклонение заявки сопровождаться обязательным комментарием администратора.

Бизнес также не дал точного ответа, можно ли пользователю отменить только что созданную заявку до её рассмотрения.

Эти вопросы специально не нужно автоматически додумывать. Во время планирования команда должна определить, какие из них блокируют оценку, какие можно вынести в отдельное исследование, а какие можно решить позднее без риска для начала разработки.

Дополнительное изменение условий

После начала обсуждения Product Owner сообщает ещё одно требование.

Первый рабочий вариант функции возврата желательно показать на внутренней демонстрации уже в конце первой недели спринта.

Это не означает, что весь эпик обязательно должен быть полностью завершён за одну неделю. Команде нужно определить, какую законченную часть пользовательского сценария реально можно продемонстрировать первой.

Дополнительное ограничение

В середине спринта специалист, который лучше всех знает платёжную интеграцию, будет отсутствовать два рабочих дня.

Если работа с `PaymentClient.refund()` окажется критической зависимостью для других задач, это необходимо учитывать при планировании.

Что должно получиться после будущего планирования

По итогам будущей практики исходный эпик должен превратиться в набор достаточно небольших и понятных задач, пригодных для спринта.

Для каждой задачи потребуется определить относительную трудоёмкость, основные зависимости и существенную неопределённость.

После этого команда должна будет решить, какие задачи реально поместить в спринт с ёмкостью:

24 story points

При этом задача практики заключается не в том, чтобы обязательно поместить весь эпик в один спринт. Если после обоснованной оценки весь объём не помещается, команда должна выбрать логически завершённую часть и объяснить границу первого инкремента.

AI-агент в дальнейшей практике будет использоваться для первичной декомпозиции, поиска пропущенных работ, анализа зависимостей, выявления неопределённости и подготовки предварительных оценок. Окончательный состав спринта и итоговые оценки будут приниматься командой после проверки предложений агента.